3.X Guided Interaction Design

PedagoLoop structures the revision of existing lesson plans as a sequence of explicit teacher decisions. In an unrestricted dialogue, the connection between a proposed change, its instructional purpose, and the text it affects may become difficult to follow. A one-step rewrite may likewise combine several changes that a novice teacher must inspect together. These concerns motivate a design that makes individual issues and candidate revisions visible, without assuming that automation alone improves instructional planning. PedagoLoop divides the lesson plan into functional teaching sections and allows the teacher to review suggestions, request alternatives, and determine which changes enter the document.

The interaction separates generating a candidate from applying it. AI output remains a proposal until the teacher explicitly accepts it. Revision therefore proceeds through manageable decisions about specific passages, with successive alternatives linked to the issue that prompted them. This organization is designed to support inspection and teacher control; its effects on revision quality and teacher development remain empirical questions.

3.X.1 Section-Based Revision Workflow

The teacher begins by pasting lesson-plan text or uploading a DOCX or PDF document. The resulting text is presented for confirmation before revision begins. This step allows the teacher to inspect the textual basis of subsequent suggestions, particularly when document extraction may affect how the original content is represented. The confirmed lesson plan is divided into functional teaching sections, and the system processes one section at a time rather than rewriting the complete document.

Within the current section, PedagoLoop generates no more than two independent suggestions. Each identifies a specific issue, explains why a change is proposed, and supplies concrete revised text. Linking these components makes the proposed intervention inspectable: the teacher can consider the stated issue and the actual wording together. Section-level processing narrows attention to a local instructional problem, while the suggestion limit is intended to keep the immediate decision task manageable. The limit of two is a design choice, not an experimentally established optimum or evidence of reduced cognitive load.

Each initial suggestion can develop into its own suggestion chain. A teaching section can be completed only when every active chain has received a final Accept or Reject decision. Requesting another candidate does not satisfy this condition. The completion rule makes unresolved proposals explicit and distinguishes the end of model generation from the end of the teacher's review.

3.X.2 Teacher Feedback Mechanism

The teacher responds to the current candidate through Accept, Reject, or Custom Revision. Accept approves that candidate and applies its proposed revision to the lesson plan. Reject ends consideration of the candidate without changing the existing text. Rejection records a decision not to use the proposal; it does not, by itself, establish why the teacher declined it or whether the underlying issue is unimportant.

Custom Revision expresses a different position: the issue is worth addressing, but the proposed solution needs adjustment. The teacher provides a new instruction for improving the current candidate, and the system generates an alternative for review. This action avoids restricting feedback to a binary choice between adopting the AI's wording and abandoning the proposed change. It also preserves the distinction between requesting an alternative and approving its application. Table 1 summarizes the three actions.

Table 1. Teacher actions and their effects

Teacher action	Meaning	System response	Effect on lesson-plan text
Accept	Approve the current candidate.	Apply the revision and close the chain.	Update the relevant text.
Reject	Decline the current candidate.	Record rejection and close the chain.	Retain existing text.
Custom Revision	Address the issue with a different solution.	Use the new instruction to generate another candidate in the chain.	No change pending explicit acceptance.

3.X.3 Iterative Suggestion Chains

Repeated Custom Revision operations produce successive candidates for the same issue. These candidates form a suggestion chain rather than a set of unrelated suggestions. The recurring interaction is: Current suggestion → Teacher instruction → New candidate → Teacher decision. After examining the new candidate, the teacher may accept it, reject it, or issue another instruction. Custom Revision can be repeated until the latest candidate receives a final decision.

Only the latest candidate in a chain is actionable. This rule establishes a single current proposal and prevents an earlier alternative from being treated as the active response after a later instruction has produced a new one. Previous candidates and teacher instructions remain in the interaction history, preserving the sequence through which the proposal changed. Their retention supports traceability without making every historical alternative simultaneously available for acceptance or rejection. Throughout the chain, the lesson-plan text remains unchanged unless the teacher explicitly accepts a candidate.

As an illustrative example, suppose the AI identifies insufficient student practice and proposes adding a group activity. The teacher considers additional practice useful but requests a shorter individual activity because classroom time is limited. The system generates a new candidate addressing the same issue under that instruction. If the alternative is still too long, the teacher can request a further revision before deciding whether to accept it. This example illustrates the intended interaction, rather than an observed classroom outcome or experimental finding.

3.X.4 Context Continuity and Version Consistency

During repeated custom revision, the model receives the current relevant lesson-plan passage, the original issue, the previous candidate, and the teacher's latest instruction. Each component has a distinct role. The passage anchors generation in the document currently being revised; the original issue maintains the purpose of the chain; the previous candidate identifies the proposal being refined; and the latest instruction specifies the requested adjustment. Together, these inputs provide continuity for the immediate revision step without requiring the model to reconstruct the entire interaction.

The system retains the original lesson plan, section versions, revision-round versions, AI suggestions, teacher decisions, custom instructions, and timestamps. This record supports tracing the relationship between generated alternatives, teacher actions, and document changes. However, the complete history is not inserted into every model request. Stored history serves traceability, whereas the selected request context serves the current generation task. This separation limits repeated inclusion of superseded material; it does not imply that every earlier instruction is automatically reconsidered at each step.

Model generation is handled asynchronously so that a long generation request does not block the interface. Because the relevant document state may change before generation finishes, the system checks that the lesson-plan section and version are still current before saving the generated result. This check is designed to prevent a result produced for an older state from being carried forward as if it belonged to a newer version. Version consistency and explicit acceptance address separate requirements: a candidate must remain associated with the appropriate document state, and the teacher must authorize any resulting textual change.

3.X.5 Design Boundaries

The current mechanism records teacher feedback and supports iterative revision within suggestion chains. Custom instructions guide the generation of subsequent candidates, but the mechanism does not automatically learn a personalized policy or update model parameters. Retaining interaction records should therefore be understood as a means of documenting revision activity, rather than evidence of automatic personalization.

PedagoLoop is designed to preserve teacher control by making proposals explicit, allowing alternatives to be requested, and requiring acceptance before modifying lesson-plan text. These are properties of the interaction mechanism, not established educational outcomes. Whether the mechanism improves lesson-plan quality or supports novice teachers' professional development requires empirical evaluation. The present design description makes no claim that these benefits have already been demonstrated.